

Length generalization is often present but suppressed: sharpening attention at inference recovers latent arithmetic circuits

Author name
Independent researcher

September 2026

Abstract

Small transformers trained on multi-digit addition are known to generalize poorly to operands longer than any seen in training, and the methods that help (reversed digits, abacus embeddings, position coupling, aligned blankspace) are known to be fragile across random seeds. We report a second kind of fragility: in many runs, out-of-distribution accuracy peaks early in training and then erodes while in-distribution accuracy stays at 100%. We then show that much of what is lost, and much of what was never reached, is not the algorithm but the attention contrast needed to run it at longer lengths. Multiplying the attention logits by a constant at inference, with no retraining, takes an 11M parameter position-coupling model trained on up to 30-digit addition from 0% to 100% exact match at 200 digits. On smaller models the same knob recovers most of what erosion removed. The intervention only helps models whose weights already encode the circuit: all four seeds of that model gain at twice the training length, one reaches 200 digits, and seeds that never generalized gain nothing at any scale. We locate the circuit that is being sharpened, a ripple-carry adder that reads the carry from the model’s own previous output digit, and show with counterfactuals that it handles carry chains of any length. Attention-logit scaling has been used before as a length-extrapolation heuristic; this work supplies a controlled case where the mechanism is visible. Code, checkpoints and every figure are public.

1 Introduction

A transformer trained to add 20-digit numbers usually cannot add 25-digit numbers. The literature on this failure has produced a sequence of input formats and position encodings that push the boundary out [1, 3–5, 11], and a warning that all of them depend heavily on the random seed [11]. This paper started as a reproduction of that ladder of methods on one consumer GPU and ended somewhere else.

Three observations led there. First, when we logged out-of-distribution accuracy throughout training rather than only at the end, it peaked early and then eroded in many runs, with in-distribution accuracy pinned at 100% throughout (Section 3). Second, when we looked for the carry in a model that had generalized, we found a ripple-carry adder implemented as two attention operations: one head that gathers the two digits of the current column, and heads in later layers that read the model’s own previous output digit to recover the incoming carry (Section 4). Third, the models that failed past the training length failed gracefully, with per-digit error rising slowly, rather than collapsing.

Together these suggested that the algorithm survives in the weights and what fails at longer lengths is attention: with more same-position distractors in the sequence, a head that must pick out two tokens spreads its mass. If so, sharpening every head at inference by multiplying its logits by a

constant should help, and it should help only models that already have the circuit. Section 5 tests this. The headline: an 11M parameter position-coupling model trained on 1 to 30 digit operands, which scores 0% exact match at 100 digits as trained, scores 100% at every length from 10 to 200 digits with its logits scaled by 2. The other three seeds of the same configuration gain at 60 digits (from 0 to 65–84%) and nothing beyond 100 at any scale. On the 3.4M models, seeds whose accuracy had eroded during training recover most of it.

Scaling attention logits is not new as a trick. Chiang and Cholak [2] scale by the logarithm of sequence length and show it lets small transformers generalize on formal-language tasks; YaRN [7] applies a temperature to attention logits when extending the context window of large language models. Our contribution is not the knob but the controlled setting in which its mechanism can be seen: a task where the algorithm is known, the circuit can be located, the dilution can be measured, and the knob can be shown to restore a specific latent computation rather than to improve things in general.

2 Setup

Task. Addition of two integers with 1 to N digits each, presented as a token sequence $\$a+b=c\$$ over a 16-symbol vocabulary. Following Lee et al. [4], digits are written least significant first, so that answer digit i depends only on the two operand digits in column i and the carry from column $i - 1$. Operands are zero-padded to a common length and the answer to one more digit. Examples are generated on the fly; the model never sees the same problem twice. The loss is next-token prediction on the answer tokens and the closing delimiter only.

Position schemes. We compare learned absolute positions (*sequential*), abacus embeddings [5], in which the position id restarts at 1 for every number with a random offset during training, and position coupling [3], in which the digits of equal significance in both operands and the answer share one position id. We also implement the fixed-width aligned blankspace augmentation of an anonymous ICLR submission [1], in which every number is padded with blank tokens to a fixed width during training and at test time, so the sequence length never changes.

Models. Pre-norm decoder-only transformers with ReLU MLPs. The small model has 4 layers, width 256, 4 heads (3.4M parameters) and is trained on 1 to 20 digit operands for 50k steps of 256 examples. The large model has 6 layers, width 384, 6 heads (11M parameters) and is trained on 1 to 30 digit operands for 60k steps of 384 examples. AdamW, cosine decay, weight decay 0.1 unless stated. Every configuration is run with three seeds unless stated; the small position-coupling model with six and the large one with four.

Evaluation. Exact match on the full answer under greedy decoding, on 128 to 256 problems whose operands both have exactly n digits, for n from 5 to 100 (small) or 200 (large). We report the final checkpoint unless stated, and we log all test lengths every 2,500 steps so that accuracy during training is also available.

3 The ladder, and what erodes

Figure 1 reproduces the known picture. Every sequential-position format learns the training distribution and falls to exactly zero one digit past it. Abacus, position coupling and fixed-width blankspace extend past the training length and fail gradually; on this budget none exceeds about

$2.5\times$ the training length, and seed variance dominates the differences between methods, as Zhou et al. [11] reported. Only two of six position-coupling seeds generalize at all, with or without weight decay.

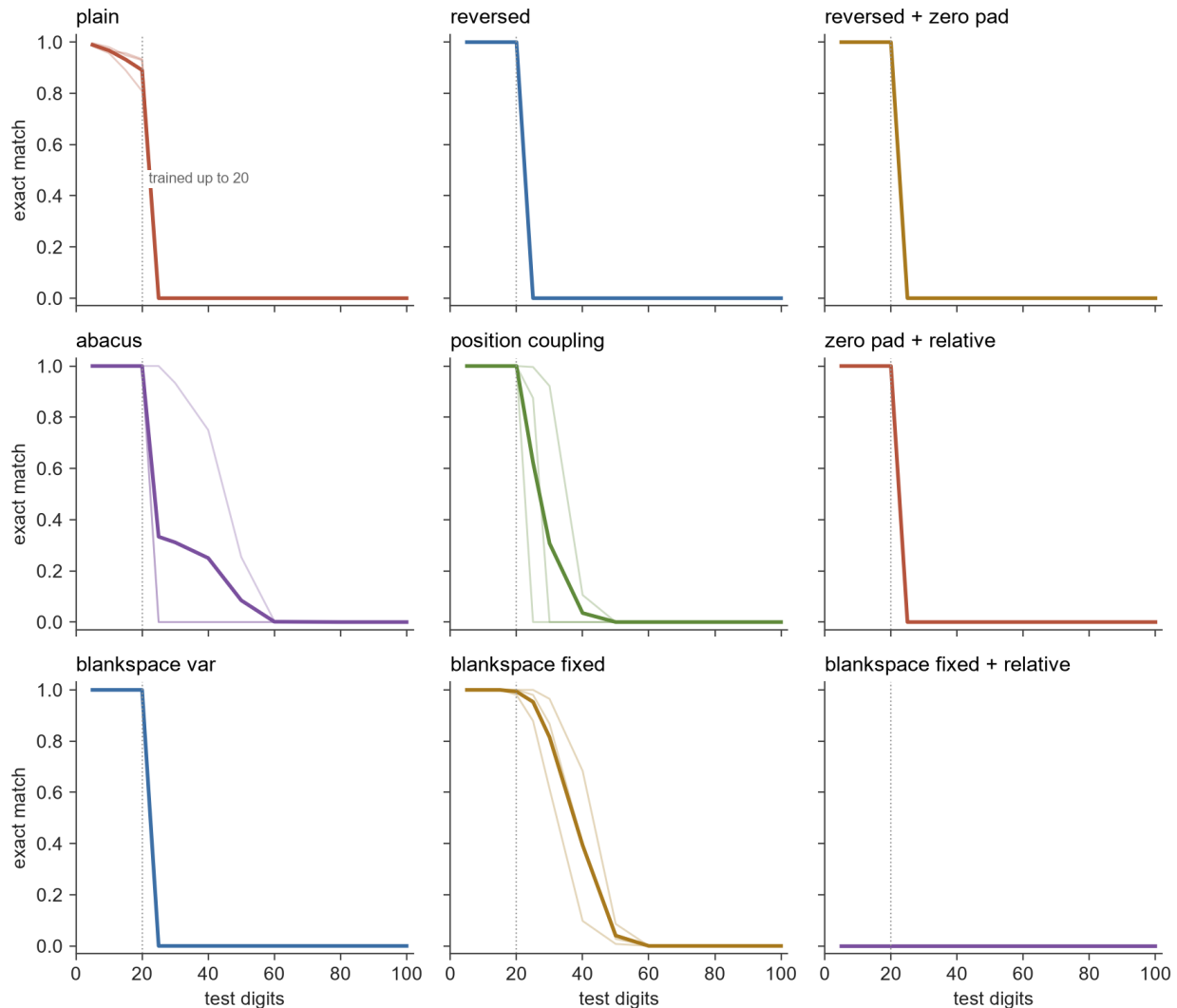


Figure 1: Exact match against test length for nine input formats on the 3.4M model trained on 1 to 20 digits. Faint lines are seeds, bold is the mean. The two blankspace variants follow an anonymous ICLR submission [1]; the relative-position rungs use a learned per-head distance bias rather than the relative embeddings of Shaw et al. [9], and the paper’s headline combination did not train under that substitution.

What the usual final-checkpoint table hides is Figure 2. In every run that generalized, out-of-distribution accuracy rose, peaked between 10k and 25k steps, and then fell, while accuracy at the training length stayed at 100% and the training loss kept decreasing. The best small position-coupling seed reached 63% at 40 digits at step 17k and finished at 11%. One large seed reached 85% at 60 digits at step 15k and finished at 75%. The erosion is not caused by the learning-rate schedule: with a constant learning rate it is worse (71% to 2%), and the cosine decay to zero

merely freezes whichever state the model is in. Without weight decay the picture is mixed: the two seeds that generalized climbed monotonically and kept what they found, but the same fraction of seeds generalized as with weight decay, so weight decay is at most a contributing factor. The methodological point stands regardless: final-checkpoint and best-checkpoint numbers for these methods can differ by 50 points.

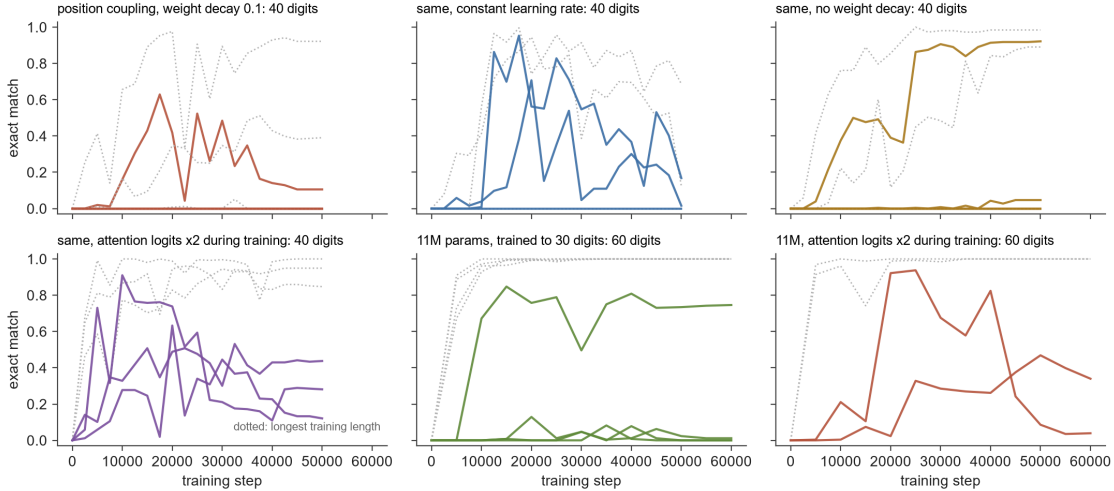


Figure 2: Out-of-distribution exact match as a function of training step for position-coupling runs, at a fixed test length past the training length; one line per seed. Dotted lines are accuracy at the longest training length. Bottom left and bottom right: models trained with their attention logits scaled by 2 throughout training.

4 The carry circuit

To know what erodes, we need to know what is there. We take the best small position-coupling model and ask where the carry comes from when it predicts answer digit i (Figure 3). Three measurements. Attention mass at the query position, averaged over answer digits, split by source: the two digits of the same column, the two digits of the column below, the model’s own previous output digit c_{i-1} , delimiters, or other digits. Teacher-forced accuracy on digit i when the carry feeding it originated k columns below and rippled through k columns summing to exactly 9. And a counterfactual: replace c_{i-1} in the prefix with the digit it would have been without its incoming carry, and count how often the prediction for c_i follows.

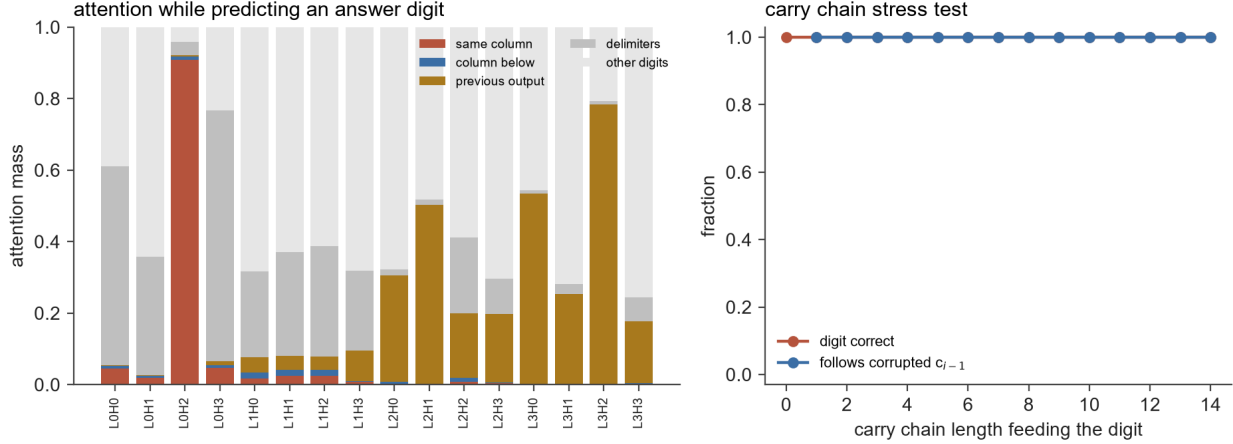


Figure 3: Left: attention by source while predicting an answer digit, per head. Right: teacher-forced accuracy against carry-chain length, and the fraction of predictions that follow a corrupted previous output digit.

The answer is a ripple-carry adder whose carry wire is the output stream. One head in layer 0 puts 91% of its attention on the two digits of the current column: it is the digit adder. Heads in layers 2 and 3 attend to the previous output digit. Whenever the column below sums to 9, so that the carry into column i depends on the carry into column $i - 1$, corrupting c_{i-1} flips the prediction for c_i 100% of the time, for chains of every length tested. The model reads its own last digit, compares it with $a_{i-1} + b_{i-1}$, and infers whether a carry arrived. Because the recursion runs through autoregression rather than through depth, chain length costs nothing: accuracy is 100% out to 14-column chains. Every operation in this circuit is a sharp attention lookup of one or two tokens.

5 Sharpening attention at inference

If the circuit is two sharp lookups, then a longer sequence, with more tokens sharing each coupled position id, should dilute them, and the failure past the training length should look like attention spreading rather than the algorithm breaking. Figure 4 (left) confirms the first part: the digit-adder head’s mass on its own column falls with test length in every model, and the seeds whose mass collapses at 30 digits are the seeds whose accuracy collapses there. The middle panel is the honest caveat: measured at the training length, that mass does not visibly soften over training, in either weight-decay setting, so the erosion of Section 3 is not simply this head going blurry. Whatever softens is somewhere else in the circuit, or shows only at longer lengths.

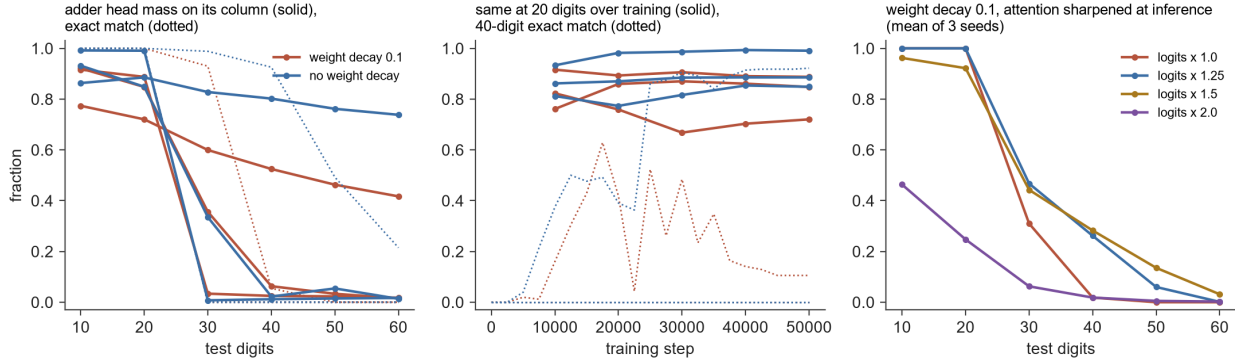


Figure 4: Left: the digit-adder head’s attention mass on its column (solid) and exact match (dotted) against test length, final checkpoints of six small position-coupling runs. Middle: the same mass at 20 digits across training, with 40-digit exact match. Right: exact match against length when every head’s logits are scaled at inference, mean of three seeds.

The intervention is a single scalar. Before the softmax, every head’s logits are multiplied by s . Nothing is retrained. Table 1 shows the small position-coupling seeds. Seed 0, which had eroded from 63% to 5% at 40 digits, returns to 89% at $s = 1.4$ with in-distribution accuracy untouched. Seed 1, which never reached 40 digits, goes from 0 to 46% at 30. Seed 2 never generalized and gains nothing. The seed that had trained without weight decay and kept its generalization also gains, from 49% to 73% at 50 digits.

seed	logits $\times$	30 digits	40	50	60
0	1.0	0.93	0.05	0	0
0	1.4	1.00	0.89	0.34	0.04
1	1.0	0	0	0	0
1	1.4	0.46	0	0	0
2	any	0	0	0	0

Table 1: Exact match for the final checkpoints of the 3.4M position-coupling seeds (weight decay 0.1) with attention logits scaled at inference.

Then the large model (Figure 5, Table 2). Seed 1, trained on up to 30 digits, scores 69% at 60 digits and 0% at 80 and beyond as trained. At $s = 2.0$ it scores 100% exact match at every length from 10 to 200 digits, on 128 problems per length. That is $6.7\times$ the training length, matching the best figure reported for this method by Cho et al. [3], from a checkpoint that scored zero at $2.7\times$. In-distribution accuracy is untouched at $s = 2.0$ and slips at $s = 2.5$. The other three seeds make the result interpretable: each gains at 60 digits (seed 0 from 0 to 84% at $s = 1.2$, seeds 2 and 3 from 0 to 71% and 65% at $s = 1.4$), none gains beyond 100 at any scale, and strong scaling destroys them. The knob amplifies the circuit that is present. In seed 1 that circuit was already the complete algorithm and needed only its attention sharpened; in the others it was not.

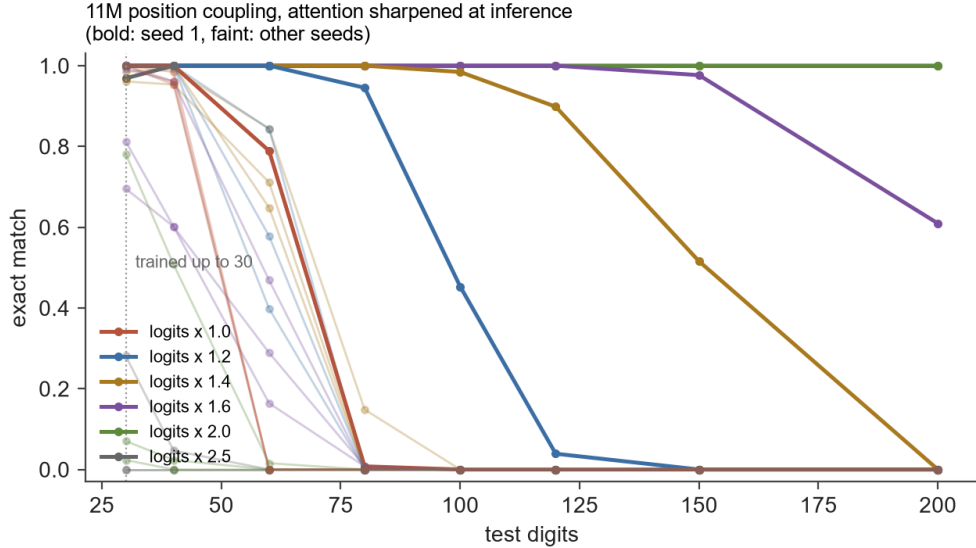


Figure 5: Exact match against test length for the 11M position-coupling models with attention logits scaled at inference. Bold: seed 1. Faint: other seeds. Trained on 1 to 30 digits.

seed 1, logits $\times$	60	80	100	120	150	200
1.0	0.69	0	0	0	0	0
1.4	1.00	1.00	0.99	0.93	0.48	0
1.6	1.00	1.00	1.00	1.00	0.97	0.59
2.0	1.00	1.00	1.00	1.00	1.00	1.00
2.5	1.00	1.00	1.00	1.00	1.00	1.00

Table 2: The 11M position-coupling model, seed 1, trained on up to 30 digits. Exact match on 128 problems per length. At $s = 2.5$ accuracy at 10 digits drops to 0.90.

How general is the recovery? Figure 6 applies the same sweep to every run in the study, at $2\times$ and $3\times$ each run’s training length. Every position-coupling seed that ever generalized gains, some by a large margin: the small seed 0 goes from 6% to 88% at 40 digits, the large seed 0 from 0 to 84% at 60. The one small abacus seed that generalized gains (70% to 88% at 40 digits, 0 to 55% at 60), and both large fixed-blankspace seeds gain at 40 digits (59% and 63% to 90% and 86%). Seeds that never generalized, in any method, gain nothing at any scale; the small blankspace models, whose sequence length does not change with operand length by construction, do not move. The knob is not free accuracy. It reads out a circuit that training produced and left too soft to use, and it does so across the position schemes that have such a circuit.

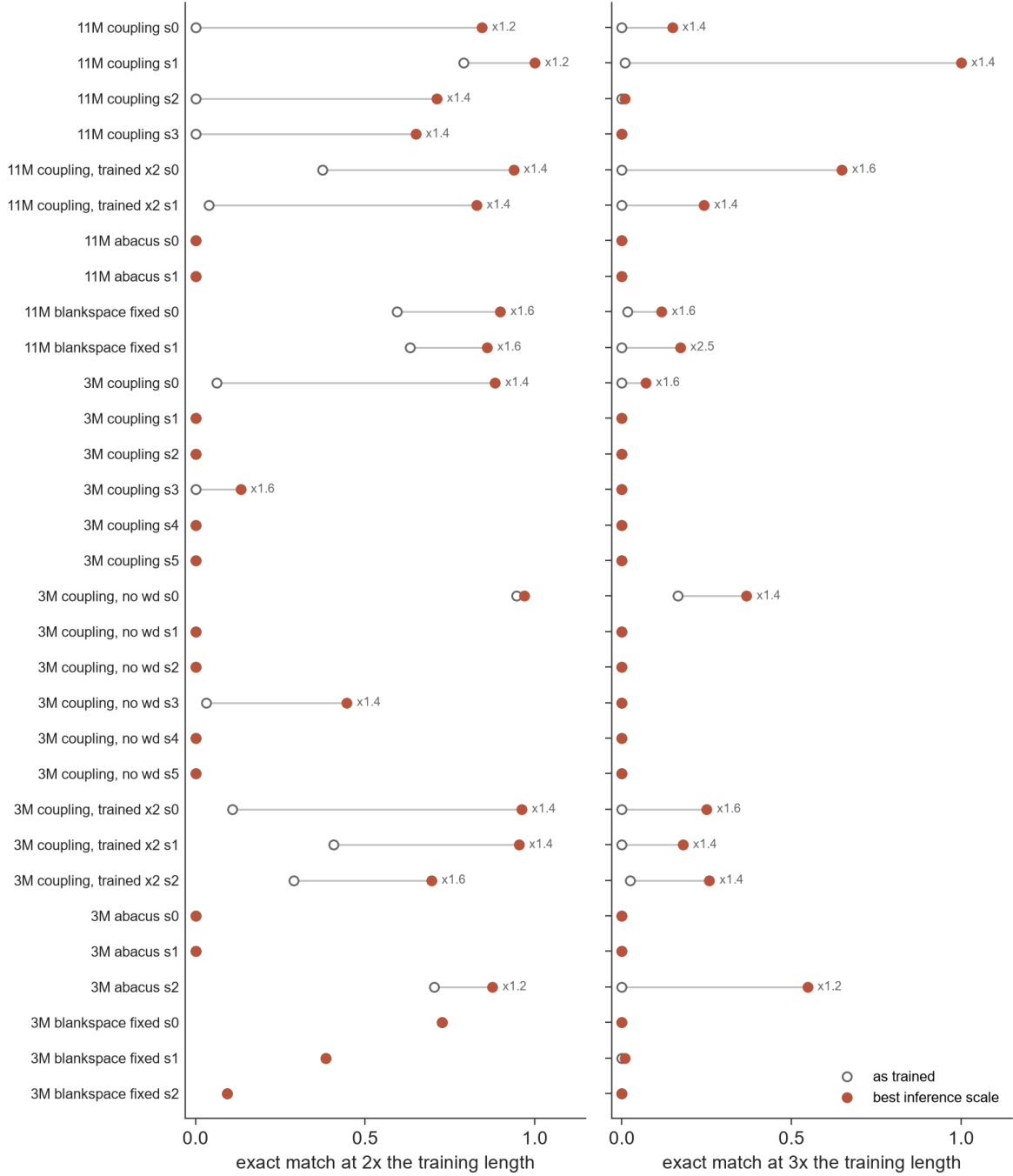


Figure 6: Exact match at $2\times$ (left) and $3\times$ (right) the training length for every run, as trained (open) and at the best inference scale (filled), with that scale marked when the gain is at least 5 points. Best scale chosen per run and per length from $\{1, 1.2, 1.4, 1.6, 2, 2.5\}$, applied on top of whatever scale the run was trained with (2 for the rows marked “trained x2”, 1 otherwise).

Where the sharpening acts. Scaling every head is crude, so Figure 7 scales subsets. On the seed that reaches 200 digits, sharpening layer 0 alone reproduces the full effect (100% at 100 digits, 98% at 150), sharpening the first two layers is indistinguishable from sharpening all six, and

sharpening every layer except layer 0 recovers nothing past the training length. Within layer 0, no single head suffices, and scaling some heads alone (H0, H3, H5) breaks the model at 60 digits, so what is being sharpened is the joint attention pattern of the first layer rather than one head. This is the layer in which the small model’s digit-adder head lives (Section 4), and it is consistent with the dilution account: the first layer is where the two operand digits of the current column are gathered from among all the tokens that share their position id, and that is the lookup that a longer sequence dilutes. On the seed that stops at 60 digits, no subset helps, as before.

11M position coupling, attention logits x2 in a subset of layers (bold: seed 1, faint: seed 0)

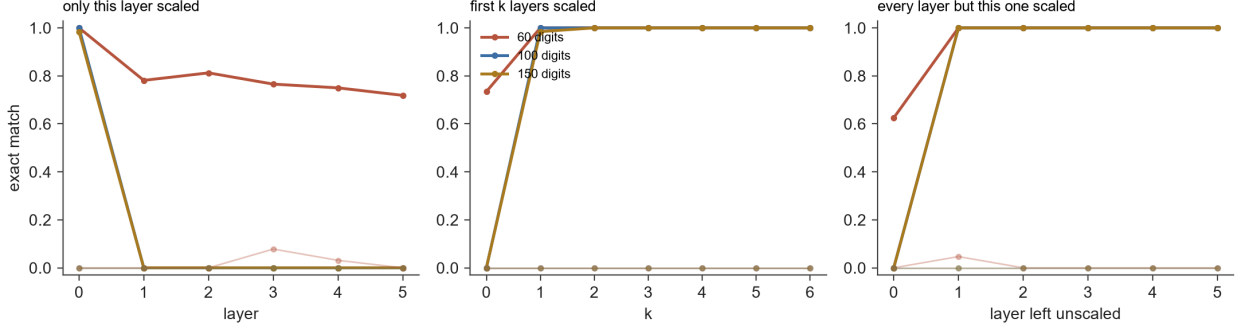


Figure 7: Exact match past the training length for the 11M position-coupling models when the attention logits are scaled by 2 in a subset of layers. Left: one layer at a time. Middle: the first k layers; $k = 0$ is the trained model and $k = 6$ is every layer. Right: every layer except one. Bold: the seed that reaches 200 digits with all layers scaled. Faint: the seed that does not.

Sharpening during training. If softness is the problem, a fixed temperature in the forward pass during training might prevent it. We trained position-coupling models with every head’s logits multiplied by 2 throughout training, three small seeds and two large. On the small model this made generalization more reliable: all three seeds reach 85–100% at 30 digits, where two of six ordinary seeds do, and two of three exceed 40% at 40 digits, where one of six ordinary seeds does. It did not prevent the erosion (Figure 2, bottom left and bottom right): the best small seed peaked at 91% on 40 digits at step 10k and finished at 28%, and one large seed peaked at 92% on 60 digits and 32% on 80 at step 20k and finished at 4% and 0. Whatever the training dynamics are doing to these circuits, a constant temperature does not stop it; it makes the circuit more likely to form and leaves it just as likely to be lost. The two knobs do stack, and the training-time one changes who benefits from the inference-time one. Applying a further factor of 1.4 to 1.6 on top of the trained temperature (Figure 6, rows marked “trained x2”) takes all five temperature-trained seeds from 4–41% to 70–96% at twice the training length, and the best large seed from 0 to 65% at 100 digits. Every temperature-trained seed is recoverable; among ordinarily trained seeds only the minority that generalized at some point are.

6 Related work

Grokking [6, 8] and its mechanistic account on modular addition are the reason this project began; we reproduce the claim of Swaroop [10] that the Fourier structure is present before generalization, and extend it to a transformer, but that result is separate from the one reported here and is described in the repository. The length-generalization ladder follows Lee et al. [4], Zhou et al. [11],

McLeish et al. [5], Cho et al. [3] and an anonymous ICLR submission [1]. The observation that a model’s per-digit rule is local and that only indexing fails at length is made in Lee et al. [4] and an anonymous ICLR submission [1]; we add the counterfactual showing that the carry is read from the model’s own output and that chain length is free.

Attention-logit scaling as a length-extrapolation device appears in Chiang and Cholak [2], who scale by $\log n$ and prove that it is needed for self-attention to represent certain formal languages at all lengths, and in YaRN [7], which applies a temperature when extending the context of rotary-position models. Our setting differs in three ways that make it a mechanism study rather than a method: the model uses learned position ids, not rotary or none; the factor is a constant chosen after training rather than a function of length; and the task allows the sharpened circuit to be located and its dependence on the intervention to be tested per seed.

7 Limitations and what we would do next

The 200-digit result is one seed of four: the other three gain from the same intervention at twice the training length and no further. What separates that seed from the others is not known. The dilution measurement in Figure 4 explains the failure with length but not the erosion during training, which remains unexplained. The relative-position variants use a per-head distance bias rather than the relative embeddings of Shaw et al. [9], and the blankspace paper’s headline combination did not learn under that substitution; a faithful implementation is future work. Everything was run on a single consumer GPU with models under 11M parameters and at most six seeds per configuration.

Two questions follow directly. The erosion is now characterized but not explained: it survives a constant learning rate, a constant attention temperature, and in some seeds the removal of weight decay, and the first-layer sharpness that the inference knob restores does not visibly soften during training when measured at the training length. Something else in the circuit degrades. And the sharpening factor is a constant chosen after the fact; a length-dependent schedule in the spirit of Chiang and Cholak [2], or a learned per-head temperature, might reach further or generalize across seeds. Both are cheap on the checkpoints we release.

Reproducibility. Code, configurations and every figure are at <https://github.com/Supergoatscriptguy/carrybit>. Final checkpoints of all 46 runs are at <https://huggingface.co/SuperGoatScriptGuy/carrybit>. Every number in this paper is produced by a script in the repository’s `experiments/` folder.

References

- [1] Anonymous. Data augmentations for arithmetic length generalization. OpenReview submission UZovxtIlym, under review at ICLR 2026, 2025.
- [2] David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [3] Hanseul Cho, Jaeyoung Cha, Pranjal Awasthi, Srinadh Bhojanapalli, Anupam Gupta, and Chulhee Yun. Position coupling: Improving length generalization of arithmetic transformers using task structure. *arXiv preprint arXiv:2405.20671*, 2024.

- [4] Nayoung Lee, Kartik Sreenivasan, Jason D Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. *arXiv preprint arXiv:2307.03381*, 2023.
- [5] Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers can do arithmetic with the right embeddings. *arXiv preprint arXiv:2405.17399*, 2024.
- [6] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023.
- [7] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.
- [8] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- [9] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of NAACL-HLT*, 2018.
- [10] Anand Swaroop. Latent algorithmic structure precedes grokking: A mechanistic study of relu mlps on modular arithmetic. *arXiv preprint arXiv:2603.23784*, 2026.
- [11] Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers can achieve length generalization but not robustly. *arXiv preprint arXiv:2402.09371*, 2024.